

Árboles Heap en Estructura de Datos

¿Qué es un Árbol Heap?

Un **árbol heap** es una estructura de datos especializada que cumple la propiedad de heap:

- En un **max-heap**, cada nodo es mayor o igual que sus hijos.
- En un **min-heap**, cada nodo es menor o igual que sus hijos.

Esto implica que:

- En un max-heap, el valor más grande está en la raíz.
- En un min-heap, el valor más pequeño está en la raíz.

Características

1. Es un **árbol binario completo**: todos los niveles están llenos excepto posiblemente el último, que se llena de izquierda a derecha.
2. Cumple la **propiedad de heap**.
3. Se puede representar fácilmente con un **arreglo**.

Representación con Arreglos

Para un nodo en la posición i del arreglo:

- Hijo izquierdo: $2i + 1$
- Hijo derecho: $2i + 2$
- Padre: $\left\lfloor \frac{i-1}{2} \right\rfloor$

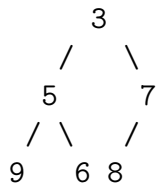
Ejemplo con un Arreglo Real

Supongamos el siguiente arreglo que representa un min-heap:

[3, 5, 7, 9, 6, 8]

Array: [3, 5, 7, 9, 6, 8]

Árbol:



- Posición 0: valor 3
 - Hijo izquierdo: $2(0) + 1 = 1$ (valor 5)
 - Hijo derecho: $2(0) + 2 = 2$ (valor 7)
 - No tiene padre (es la raíz).
- Posición 1: valor 5
 - Padre: $\left\lfloor \frac{1-1}{2} \right\rfloor = 0$ (valor 3)
 - Hijo izquierdo: $2(1) + 1 = 3$ (valor 9)
 - Hijo derecho: $2(1) + 2 = 4$ (valor 6)
- Posición 2: valor 7
 - Padre: $\left\lfloor \frac{2-1}{2} \right\rfloor = 0$ (valor 3)
 - Hijo izquierdo: $2(2) + 1 = 5$ (valor 8)
 - Hijo derecho: $2(2) + 2 = 6$ (*no existe, fuera del arreglo*)
- Posición 5: valor 8
 - Padre: $\left\lfloor \frac{5-1}{2} \right\rfloor = 2$ (valor 7)
 - Hijo izquierdo: $2(5) + 1 = 11$ (*fuera del arreglo*)
 - Hijo derecho: $2(5) + 2 = 12$ (*fuera del arreglo*)

Ventajas de esta Representación

- No se requieren punteros o referencias explícitas.
- Navegación eficiente en tiempo constante entre padre e hijos.
- Uso óptimo de memoria gracias al uso de un arreglo contiguo.

Operaciones Básicas

Insertar

1. Insertar al final del arreglo.
2. Aplicar *heapify up*: intercambiar con el padre si se viola la propiedad.

Eliminar la raíz

1. Reemplazar raíz con el último elemento.
2. Eliminar el último.
3. Aplicar *heapify down* desde la raíz.

Heapify

Proceso para restaurar la propiedad del heap desde un nodo hacia abajo.

Aplicaciones

- Colas de prioridad
- HeapSort
- Dijkstra
- Compresión de Huffman

Complejidad Computacional

Operación	Complejidad
Insertar	$O(\log n)$
Eliminar raíz	$O(\log n)$
Obtener raíz	$O(1)$
Heapify	$O(\log n)$
Construir heap desde array	$O(n)$

Tipos de Heap

- Min-Heap
- Max-Heap
- Binomial Heap

- Fibonacci Heap
- Binary Heap

Comparación con Árboles Binarios de Búsqueda (BST)

Característica	Heap	BST
Ordenamiento	Parcial	Total
Búsqueda	Ineficiente	Eficiente
Acceso a extremo	$O(1)$ (raíz)	$O(\log n)$ promedio
Uso principal	Prioridad	Búsqueda y orden